CS102: Data Structures and Algorithms

Week 4: Searching, Sorting, & Asymptotic Growth Mechanics

Milestone 1: Search Topologies & Linear Iteration Space

In computer science, a search problem consists of an active target node situated within an indexed data collection frame. The fundamental mechanics of searching require evaluating structural configurations to optimize retrieval performance.

Lessons 1 & 2: The Linear Search Paradigm & Target-Collection Mechanics

A basic linear search traverses an unsorted collection sequentially from left to right, comparing each node directly against the target. The operational workload is highly dependent on where the element resides within the collection.

■ Expanded Content: Structural Target Positions

- **Best Case (First Offset):** Target is located at index 0. Operational burden is exactly 1 step lookup comparison, completely independent of collection size.
- **Worst Case (Final Offset / Missing):** Target is located at the final tail offset or is entirely absent from the collection. The loop must execute exactly n sequential comparisons.

```
# Simulation: Sequential Linear Search with Explicit Operation Telemetry
def linear_search_telemetry(collection, target):
    comparisons = 0
    found_flag = False
    for item in collection:
        comparisons += 1
        if item == target:
            found_flag = True
            break # Early exit protocol to optimize processing loops
    return {"found": found_flag, "total_comparisons": comparisons}
```

Milestone 2: Asymptotic Search Scaling & Binary Splitting

As arrays expand to hold massive data sets, linear scanning becomes highly inefficient. Shifting to logarithmic performance profiles requires enforcing pre-sorted memory invariants.

Lessons 3 & 4: Binary Search & The Logarithmic Halving Blueprint

The Binary Search algorithm requires that the target collection be pre-sorted. By maintaining two pointer offsets, it calculates a middle division node. If a match is not found immediately, it discards exactly half of the remaining address space, dropping time complexity down to a highly sustainable profile.

```
# Safe Binary Search Implementation under Asymptotic O(log n) Performance
def binary_search_trace(sorted_data, target):
    low = 0
    high = len(sorted_data) - 1
    steps = 0
    while low <= high:
        mid = (low + high) // 2
        steps += 1
        if sorted_data[mid] == target:
            return {"index": mid, "executed_steps": steps}
        elif sorted_data[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return {"index": -1, "executed_steps": steps}
```

Lessons 5 & 6: Master Reference Matrix — Linear vs. Logarithmic Performance

The architectural decision matrix between linear search and binary search highlights why pre-sorting data is vital for scalable software engineering.

Collection Scale (n)	Linear Search Cost (O(n))	Binary Search Cost (O(log n))	Selection Rule
n = 10 Elements	`10 Comparisons`	`~3–4 Comparisons`	Simplest path for small unsorted structures.
n = 1,000 Elements	`1,000 Comparisons`	`~10 Comparisons`	Clear win if data is pre-sorted.
n = 1,000,000 Elements	`1,000,000 Comparisons`	`~20 Comparisons`	Binary search mandatory for scale.

Milestone 3: Sorting Approaches & Composite Data Contracts

Sorting transitions data from chaotic sets into structured arrays, enabling advanced reporting and clean analytical summaries.

Lessons 7–10: Bubble Sort vs. Insertion Sort Critical Architectures

Classic sorting approaches vary significantly in performance, particularly when handling partially pre-sorted datasets.

■ Expanded Content: Quadratic Sorting Paradigms

- **Bubble Sort (Neighbor-Swapping):** Iteratively scans a collection, swapping adjacent pairs if they violate the sorted order. This forces the largest values to bubble rightward. Runs in quadratic time.

- **Insertion Sort (Left-Sliding):** Iteratively builds a sorted boundary frame on the left side of the list. It takes each new element and slides it leftward into place. Highly efficient on nearly-sorted data.

```
# Structural Reference: Insertion Sort Copy Engine (Preserves In-Place Constraints)
def insertion_sort_isolated(nums):
    nums_copy = list(nums)
    for i in range(1, len(nums_copy)):
        key = nums_copy[i]
        j = i - 1
        while j >= 0 and key < nums_copy[j]:
            nums_copy[j + 1] = nums_copy[j]
            j -= 1
        nums_copy[j + 1] = key
    return nums_copy
```

Lessons 13 & 14: Native Python Sorting & Composite Stable Contracts

Production-ready applications rely on Python's built-in sorting tools. Python's sort is guaranteed to be stable, meaning elements with equal keys maintain their original relative order.

```
# Production Strategy: Multi-Criteria Leaderboard Sorting using Composite Keys
leaderboard_data = [
    {"name": "Ava", "score": 100, "level": 3},
    {"name": "Ben", "score": 150, "level": 2},
    {"name": "Cara", "score": 150, "level": 5}
]

# Rule: Sort by score descending, then by level descending, then by name ascending
calibrated_leaderboard = sorted(
    leaderboard_data,
    key=lambda item: (-item["score"], -item["level"], item["name"])
)
```